

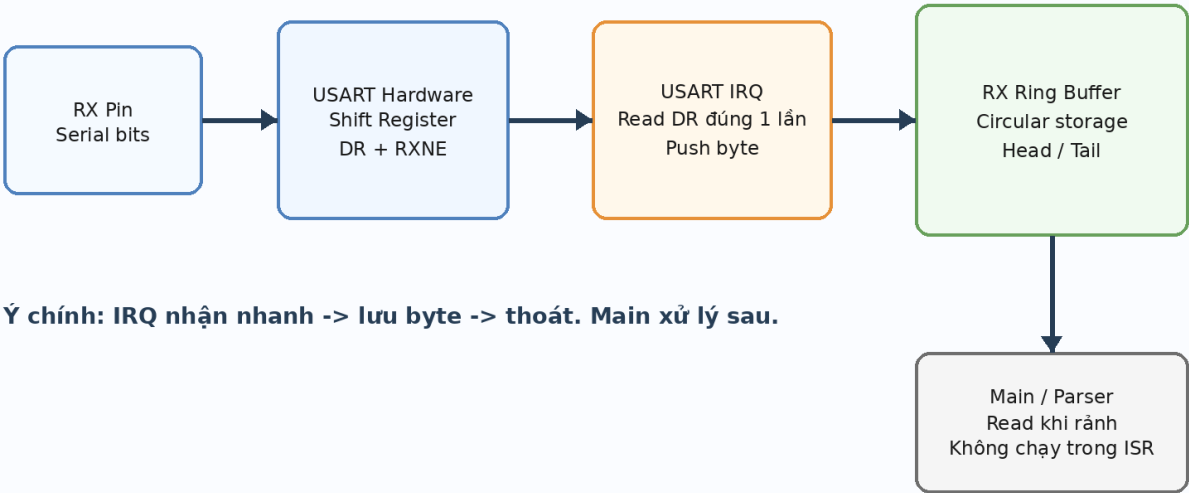
SOFTWARE DESIGN SPECIFICATION

# USART + RING BUFFER

## STM32F103xB Bare-Metal Driver

Mục tiêu: đọc xong là hiểu vì sao USART cần Ring Buffer, Ring Buffer quay vòng như thế nào, và ISR/Main phối hợp ra sao.

### USART + Ring Buffer: Bức tranh tổng thể

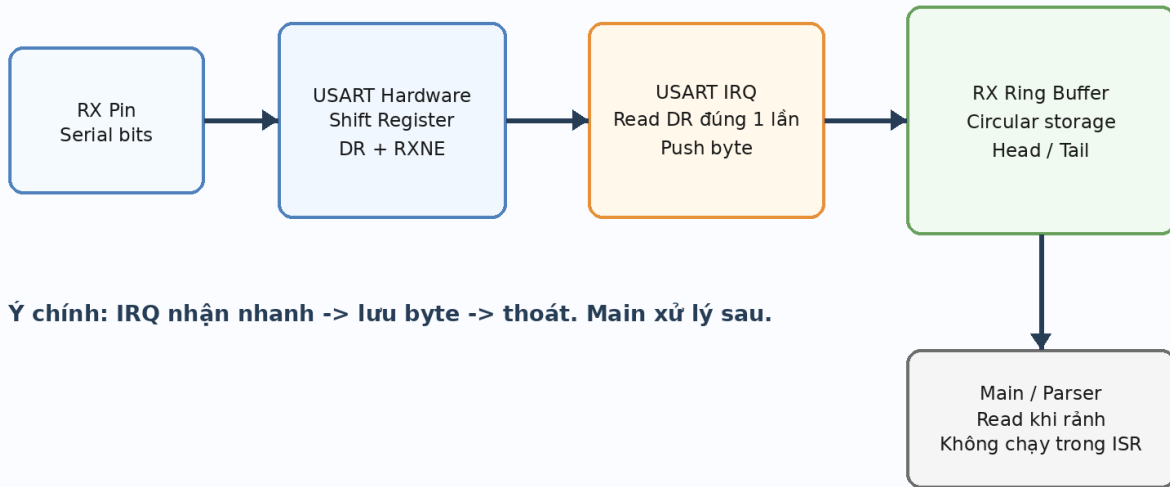


|              |                                       |
|--------------|---------------------------------------|
| Project      | STM32F103_BareMetal_Drivers           |
| Modules      | hal_uart.c/.h + hal_ringbuffer.c/.h   |
| Target       | STM32F103xB / Cortex-M3               |
| Design model | USART RX interrupt + SPSC Ring Buffer |
| Revision     | 1.0 - 04/09/2026                      |

# 1. Tại sao USART cần Ring Buffer?

USART nhận dữ liệu theo thời gian thực. Byte mới có thể đến bất kỳ lúc nào, trong khi main() có thể đang xử lý LCD, protocol, sensor hoặc tác vụ khác. Nếu ISR làm quá nhiều việc hoặc application chưa kịp nhận byte trước, nguy cơ mất dữ liệu tăng lên.

## USART + Ring Buffer: Bức tranh tổng thể



**Mental model: ISR chỉ làm việc khẩn cấp - lấy byte khỏi phần cứng, cất vào RAM, rồi thoát. Main xử lý nội dung sau.**

### Nếu không có Ring Buffer:

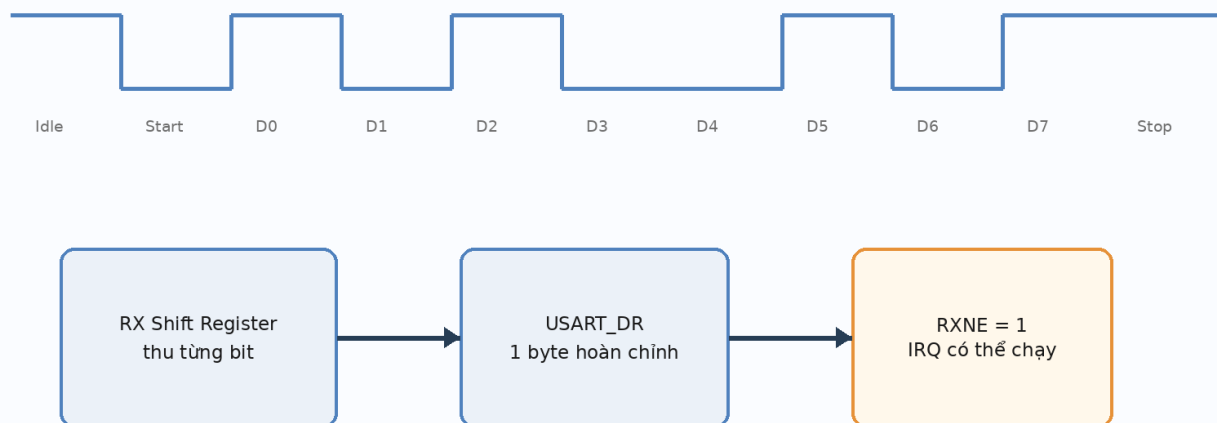
- Mỗi lần RXNE xảy ra, application phải xử lý hoặc chuẩn bị đúng buffer ngay lập tức.
- Receive\_IT kiểu fixed-length phù hợp khi biết trước chính xác cần nhận bao nhiêu byte.
- Stream liên tục, AT command, console hoặc packet có độ dài thay đổi sẽ khó quản lý hơn.

### Nếu có Ring Buffer:

- USART ISR trở thành producer: nhận byte và push nhanh vào buffer.
- main() trở thành consumer: đọc byte theo tốc độ xử lý của application.
- Khoảng chênh lệch tốc độ giữa UART và main được hấp thụ bởi dung lượng buffer.

## 2. Hiểu USART RX trước khi hiểu Ring Buffer

### USART RX: Từ chân RX đến byte trong DR



Đọc DR sẽ lấy byte đã nhận; nếu phần mềm không đọc kịp, byte sau có thể gây overrun.

Trong RX path, phần cứng thu serial bit trên chân RX, ghép thành character trong receive shift register và chuyển dữ liệu vào USART\_DR. Khi dữ liệu sẵn sàng, RXNE được set. Interrupt handler phải đọc DR để lấy byte.

### Flow đơn giản:

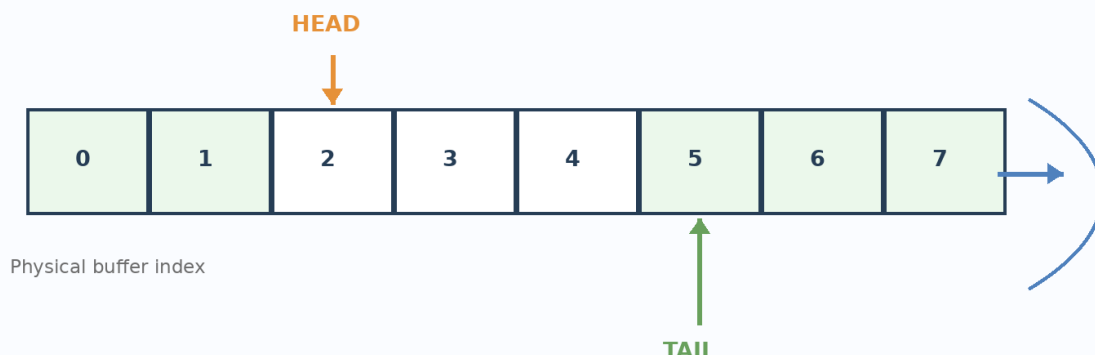
RX pin -> Receive Shift Register -> USART\_DR -> RXNE -> IRQ -> software

**Điểm quan trọng:** DR là điểm giao giữa hardware và software. Trong một RXNE event, driver nên đọc DR đúng một lần rồi quyết định đưa byte đó vào nơi nào.

Nếu byte trước chưa được software giải phóng mà byte mới đến, USART có thể gặp overrun. Ring Buffer không thay thế DR; nó giúp phần mềm rời DR thật nhanh.

### 3. Ring Buffer thực chất là gì?

#### Ring Buffer là gì? Bộ nhớ tuyến tính nhưng index quay vòng



Head/Tail tăng theo sequence counter. Index thực tế = counter & (Size-1) khi Size =  $2^N$ .

Ring Buffer dùng một mảng RAM bình thường nhưng coi đầu và cuối mảng nối với nhau. Khi index đi qua phần tử cuối, nó quay lại phần tử 0. Vì vậy dữ liệu được sử dụng theo FIFO: byte vào trước được đọc trước.

```

Producer WRITE tại Head
Consumer READ tại Tail
Count = Head - Tail

```

Trong implementation tối ưu hiện tại, Head và Tail là monotonic counter. Chúng không cần reset về 0 sau mỗi vòng. Hàm RingBuffer\_GetIndex() chuyển counter thành vị trí vật lý trong mảng.

#### Ví dụ Size = 256:

```

Mask = Size - 1 = 255
Index = Counter & 255

```

```

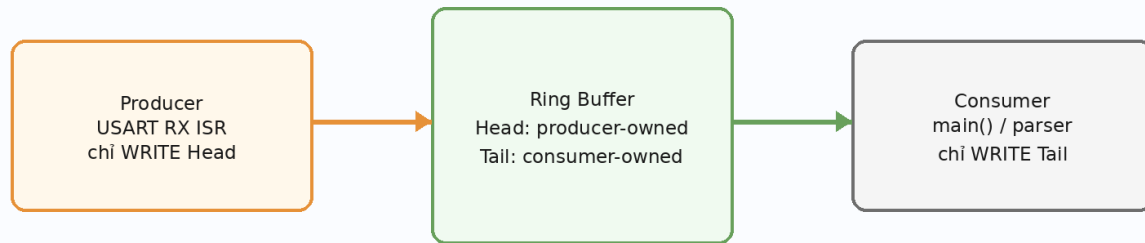
Counter: 254 255 256 257 258
Index:   254 255  0  1  2

```

**Do đó đây vẫn là ring/circular buffer thật, chỉ khác ở chỗ sequence counter tăng liên tục còn index RAM quay vòng.**

## 4. Tại sao bỏ biến Count?

### Write / Read: Producer và Consumer không đụng cùng biến



#### WriteChar()

1. snapshot Head/Tail
2. check FULL
3. `buffer[index] = Data`
4. `publish Head++`

#### ReadChar()

1. snapshot Tail/Head
2. check EMPTY
3. `Data = buffer[index]`
4. `publish Tail++`

**Không dùng biến Count chung -> giảm race condition giữa ISR và main**

Code Ring Buffer kiểu đơn giản thường lưu Head, Tail và Count. Nhưng với USART RX interrupt, ISR có thể `Count++` trong lúc `main()` `Count--`. Hai phía cùng sửa một biến shared tạo ra race condition nếu operation bị ngắt giữa chừng.

### Thiết kế SPSC giải quyết bằng ownership rõ ràng:

| Thành phần | Ai ghi?              | Ai chỉ đọc? |
|------------|----------------------|-------------|
| Head       | Producer / USART ISR | Consumer    |
| Tail       | Consumer / main()    | Producer    |

Count không phải shared state nữa; nó được tính khi cần:

```
count = (uint32_t)(Head - Tail);
```

**SPSC = Single Producer / Single Consumer. Đây chính là mô hình rất tự nhiên cho USART RX: IRQ tạo dữ liệu, main tiêu thụ dữ liệu.**

## 5. Empty, Full, Wrap-around

### Empty, Full và Count



#### Ví dụ Size = 8

Head=10, Tail=10

Count=0 -> EMPTY

Head=13, Tail=10

Count=3

Head=18, Tail=10

Count=8 -> FULL

Ưu điểm sequence-counter: dùng đủ Size byte, vẫn xử lý wrap-around uint32\_t theo unsigned arithmetic.

### Điều kiện của thiết kế hiện tại:

EMPTY: Head == Tail

COUNT: Head - Tail

FULL : (Head - Tail) >= Size

Khác với implementation dùng nextHead == Tail, thiết kế sequence-counter sử dụng được toàn bộ Size byte. Ví dụ mảng 256 byte có capacity 256 byte.

Head/Tail là uint32\_t. Khi vượt 0xFFFFFFFF, unsigned arithmetic quay vòng modulo  $2^{32}$ . Miễn chênh lệch Head - Tail luôn bị giới hạn bởi Size nhỏ hơn rất nhiều so với  $2^{32}$ , count vẫn có ý nghĩa đúng cho use case này.

## 6. Flow của RingBuffer\_WriteChar()

Đây là API quan trọng nhất trong USART ISR.

```
head = pRingBuf->Head;
tail = pRingBuf->Tail;

if ((head - tail) >= Size)
    return FULL;

index = GetIndex(head);
buffer[index] = Data;
BARRIER();
Head = head + 1;
```

### Ý nghĩa từng bước:

- Snapshot Head/Tail để giảm volatile access và giữ logic ngắn.
- Kiểm tra FULL trước khi đề lên dữ liệu chưa đọc.
- Ghi data vào RAM trước.
- Sau đó mới publish Head mới. Consumer chỉ thấy byte mới sau khi data đã được store.

Nếu Size = power-of-two, GetIndex dùng bit mask nhanh hơn phép chia lấy dư.

## 7. Flow của RingBuffer\_ReadChar()

```
tail = pRingBuf->Tail;
head = pRingBuf->Head;

if (head == tail)
    return EMPTY;

index = GetIndex(tail);
*pData = buffer[index];
BARRIER();
Tail = tail + 1;
```

Consumer đọc byte cũ nhất và chỉ sau khi đã copy data ra mới advance Tail. Sau đó vị trí RAM đó được xem là free và producer có thể tái sử dụng ở vòng sau.

### FIFO example:

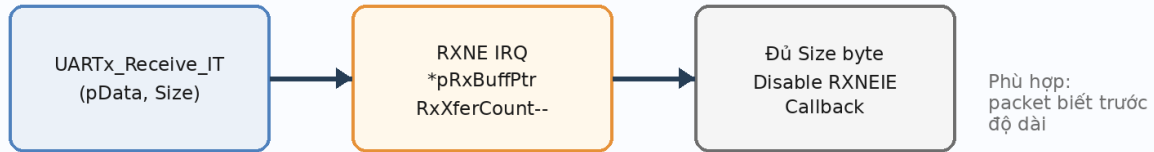
```
IRQ writes: A B C D
main reads: A -> B -> C -> D
```

**Ring Buffer không tự biết packet, newline hay AT response. Nó chỉ quản lý byte FIFO. Protocol parser nằm ở layer phía trên.**

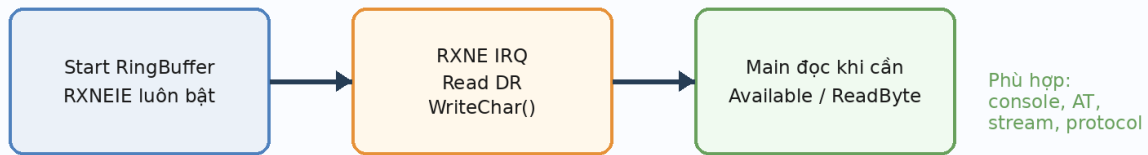
## 8. UART RX cũ vs UART RX Ring Buffer

### UART RX cũ và RX Ring Buffer khác nhau thế nào?

#### RX Interrupt cố định (code cũ)



#### RX liên tục bằng Ring Buffer



Code UART ban đầu dùng `pRxBuffPtr`, `RxXferSize` và `RxXferCount`. `UARTx_Receive_IT()` yêu cầu application cung cấp buffer và `Size`; IRQ nhận đủ `Size` byte thì disable `RXNEIE` và callback.

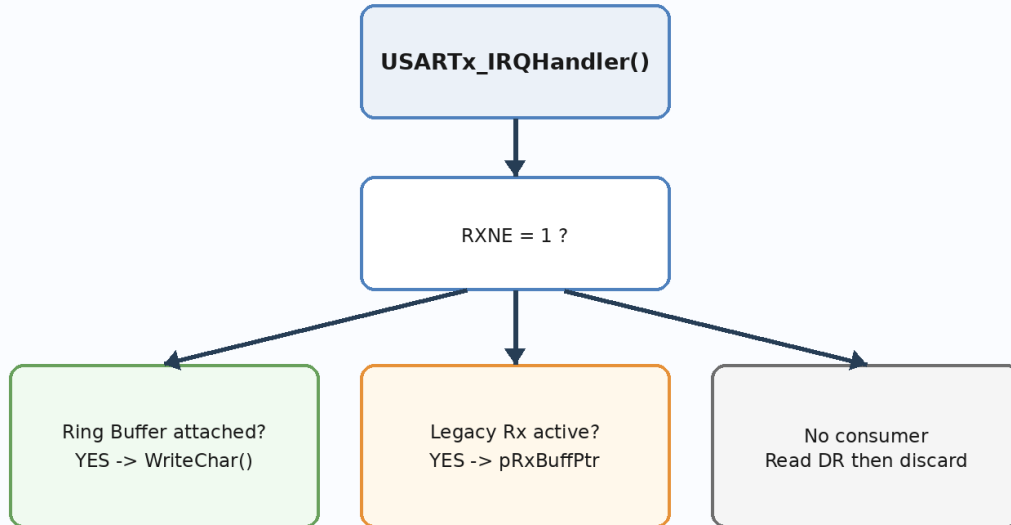
Ring Buffer không cần biết trước packet dài bao nhiêu. `RXNE` interrupt có thể bật liên tục; mỗi byte vào buffer, còn main quyết định khi nào đọc.

| Tiêu chí            | Receive_IT fixed-length          | Ring Buffer continuous      |
|---------------------|----------------------------------|-----------------------------|
| Biết trước Size     | Cần                              | Không cần                   |
| <code>RXNEIE</code> | Tắt khi đủ Size                  | Bật liên tục                |
| ISR                 | Copy vào <code>pRxBuffPtr</code> | Push 1 byte                 |
| Main                | Chờ callback/buffer              | Đọc Available/ReadByte      |
| Use case            | Frame cố định                    | AT, console, stream, parser |

## 9. Cách tích hợp Ring Buffer mà ít ảnh hưởng code UART cũ

Nếu mục tiêu là tránh regression, giữ nguyên các API cũ và chỉ thêm state/API Ring Buffer. Điểm bắt buộc phải mở rộng là RXNE branch trong USARTx\_IRQHandler(), vì USART\_DR chỉ có một nguồn byte.

### Thiết kế IRQ khi tích hợp Ring Buffer



**Nguyên tắc quan trọng: USART\_DR chỉ đọc đúng 1 lần cho mỗi RXNE event.**

### Quy tắc dispatch:

```

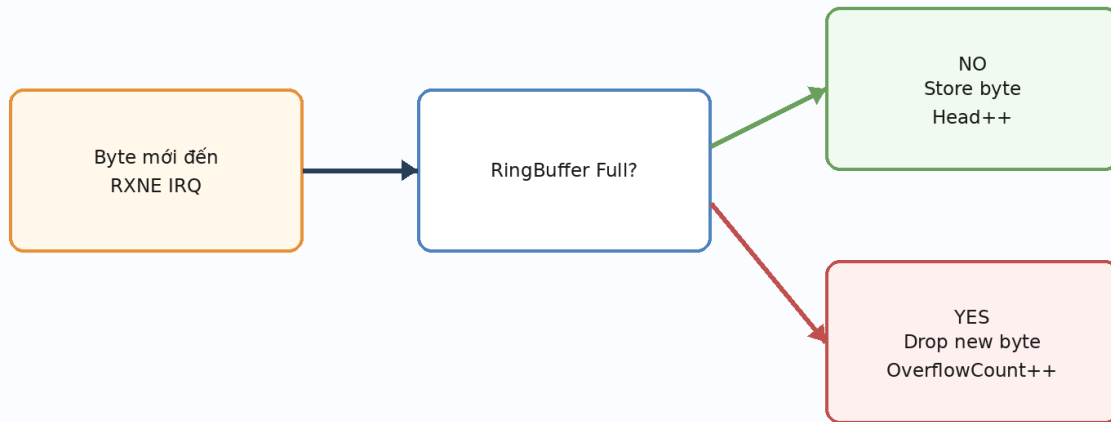
rx_data = (uint8_t)(USART_DR & 0xFF); // read once

if (pRxRingBuffer != NULL)
    RingBuffer_WriteChar(...);
else if (RxXferCount > 0)
    *pRxBuffPtr++ = rx_data;
else
    discard;
  
```

**Không để DMA RX, legacy RX\_IT và Ring Buffer RX cùng consume DR. Chỉ một RX path được active tại một thời điểm.**

## 10. Overflow: chuyện gì xảy ra nếu main quá chậm?

### Khi Ring Buffer đầy: Overflow policy



Drop-new giữ nguyên dữ liệu cũ chưa đọc. OverflowCount giúp debug xem main có xử lý kịp không.

Buffer chỉ hấp thụ chênh lệch tốc độ trong một khoảng thời gian. Nếu producer tiếp tục nhanh hơn consumer đủ lâu, buffer sẽ full.

### Policy hiện tại là drop-new:

- Giữ nguyên các byte cũ chưa được main xử lý.
- Byte mới đến khi FULL bị bỏ.
- RxOverflowCount tăng để chẩn đoán.

### Cách xử lý khi overflow tăng thường xuyên:

- Tăng RX buffer size, ưu tiên 128/256/512/1024.
- Main đọc buffer thường xuyên hơn hoặc parse nhanh hơn.
- Giảm baud rate nếu hệ thống cho phép.
- Với burst lớn hoặc throughput cao, cân nhắc USART RX DMA + circular buffer/IDLE line.

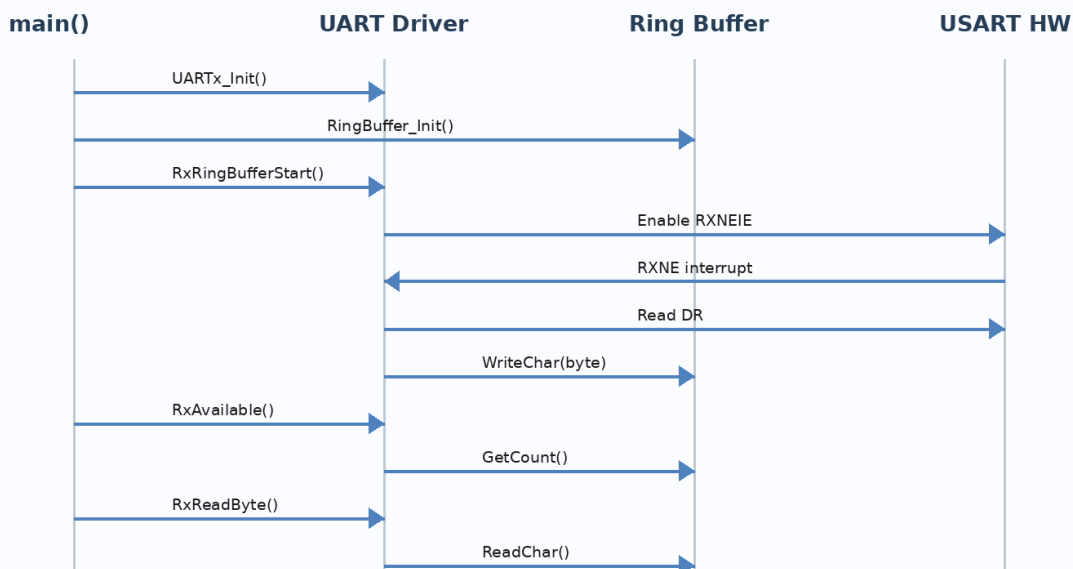
## 11. API design và responsibility

| API                      | Layer           | Ý nghĩa                                 |
|--------------------------|-----------------|-----------------------------------------|
| RingBuffer_Init          | Generic utility | Attach RAM, Size, Mask, reset Head/Tail |
| RingBuffer_WriteChar     | Generic utility | Producer push 1 byte                    |
| RingBuffer_ReadChar      | Generic utility | Consumer pop 1 byte                     |
| RingBuffer_GetCount      | Generic utility | Số byte đang chờ                        |
| UARTx_RxRingBufferStart  | UART adapter    | Attach ring + enable RXNEIE             |
| UARTx_RxReadByte         | UART adapter    | Read 1 byte từ attached ring            |
| UARTx_RxAvailable        | UART adapter    | Unread bytes                            |
| UARTx_IRQHandler         | UART core       | Read DR và dispatch byte                |
| UARTx_GetRxOverflowCount | UART diagnostic | Theo dõi byte bị drop                   |

Generic Ring Buffer không nên biết USART. UART layer phụ thuộc Ring Buffer, không ngược lại. Nhờ đó Ring Buffer có thể dùng lại cho SPI stream, logging queue hoặc protocol khác.

## 12. Sequence sử dụng trong main()

### Sequence khởi tạo và chạy thực tế



```

static RingBuffer_HandleTypeDef uart1RxRing;
static uint8_t uart1RxMem[256];

UARTx_Init(&huart1);
RingBuffer_Init(&uart1RxRing, uart1RxMem, sizeof(uart1RxMem));
UARTx_RxRingBufferStart(&huart1, &uart1RxRing);

while (1)
{
    uint8_t ch;
    if (UARTx_RxReadByte(&huart1, &ch) == RING_BUFFER_OK)
    {
        Protocol_ProcessByte(ch);
    }
}
  
```

### IRQ vector:

```

void USART1_IRQHandler(void)
{
    UARTx_IRQHandler(&huart1);
}
  
```

**Main không cần gọi `UARTx_Receive_IT()` lặp đi lặp lại. Ring Buffer RX chạy liên tục cho đến khi Stop.**

## 13. Ví dụ AT command thực tế

Giả sử module trả về:

```
0 K \r \n
```

UART ISR nhận từng byte:

```
'0' -> ring
'K' -> ring
'\r' -> ring
'\n' -> ring
```

Main có thể đọc byte và xây line parser:

```
while (UARTx_RxReadByte(&huart1, &ch) == RING_BUFFER_OK)
{
    if (ch == '\n')
    {
        ParseLine(lineBuffer);
        lineIndex = 0U;
    }
    else
    {
        lineBuffer[lineIndex++] = ch;
    }
}
```

Đây là lý do Ring Buffer đặc biệt hợp với Wi-Fi/Bluetooth module dùng AT command: response có thể đến bất kỳ lúc nào và độ dài không cố định.

## 14. Verification / Test Plan

### Test plan tối thiểu để tin tưởng thiết kế

|         |                            |                         |
|---------|----------------------------|-------------------------|
| RB-01   | Write 1 byte / Read 1 byte | Data đúng               |
| RB-02   | Wrap qua cuối mảng         | Index quay về 0         |
| RB-03   | Fill đúng Size byte        | FULL đúng lúc           |
| RB-04   | Read khi empty             | EMPTY, không đổi Tail   |
| UART-01 | RXNE từng byte             | Byte vào ring           |
| UART-02 | Burst > tốc độ main        | Không ORE nếu buffer đủ |
| UART-03 | Buffer full                | OverflowCount tăng      |
| UART-04 | Ring + legacy mode         | Không dùng đồng thời    |

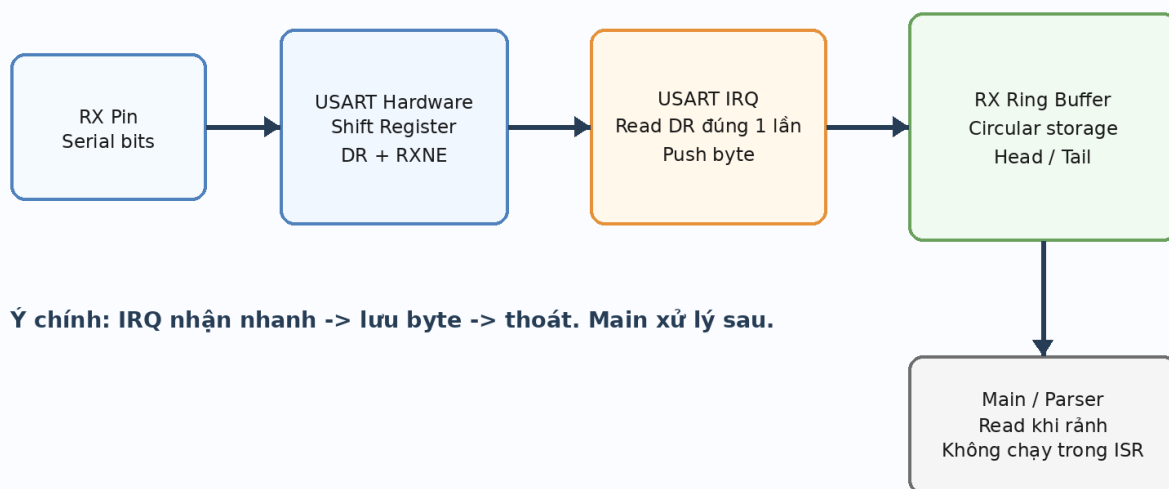
Ngoài test functional, nên dùng logic analyzer hoặc oscilloscope để xác nhận baud/frame và test burst data ở tốc độ cao. Với Ring Buffer, test quan trọng nhất là wrap-around và tình huống ISR ghi trong lúc main đang đọc.

### Test loop đơn giản:

1. Initialise ring Size = 8
2. Write 0..7 -> expect FULL after 8 bytes
3. Read 4 bytes -> expect 0,1,2,3
4. Write 8..11 -> data wraps to indexes 0..3
5. Read remaining -> expect 4,5,6,7,8,9,10,11

## 15. Những điều cần nhớ

### USART + Ring Buffer: Bức tranh tổng thể



- USART hardware nhận bit; RXNE báo DR có byte.
- ISR phải đọc DR nhanh, không parse protocol dài trong interrupt.
- Ring Buffer là FIFO quay vòng trong RAM.
- Producer chỉ sửa Head; Consumer chỉ sửa Tail.
- Empty = Head == Tail; Count = Head - Tail; Full khi Count >= Size.
- Power-of-two Size cho phép index bằng bit-mask nhanh.
- Ring Buffer chỉ chứa byte; application/protocol quyết định ý nghĩa byte.
- OverflowCount > 0 nghĩa là consumer không theo kịp producer.
- RX Ring Buffer, fixed-length RX\_IT và DMA RX không nên cùng consume USART\_DR.

**Một câu để nhớ: USART IRQ lấy byte khỏi hardware càng nhanh càng tốt; Ring Buffer giữ byte an toàn; main xử lý khi có thời gian.**